

Railway LiveKit Deploy — make the autoscaler actually add/remove nodes

How to stand up a scalable LiveKit cluster on Railway so that setting `LIVEKIT_SCALE_PROVIDER=railway` genuinely grows and shrinks the media tier as viewers come and go.

What's already done in code: `livekitScaleController` reads concurrent viewers every minute, computes the node count, and calls Railway's `serviceInstanceUpdate` API to set the LiveKit service's replica count — scaling out under load and in to zero when idle. That half needs no more work. **What this guide does:** deploy the LiveKit service (so there's something to scale) and hand the scaler the four values it needs.

You do the account/credential steps yourself (I don't create accounts, enter secrets, or generate tokens). Everything below is a one-time setup; after it, scaling is automatic.

Step 0 — Generate an API key/secret pair

On your machine, generate a random key and secret (any 2 random strings; these must match on the LiveKit service and the app):

```
echo "LIVEKIT_API_KEY=API$(openssl rand -hex 6)"
echo "LIVEKIT_API_SECRET=$(openssl rand -hex 32)"
```

Keep both. They go in two places (the LiveKit service and the app), and nowhere else.

Step 1 — Deploy the LiveKit service on Railway

1. In your Railway project, **New → Deploy from Repo**, point it at this repo and set the service's **root directory** to `deploy-kit/livekit` (it has the `Dockerfile` + `entrypoint.sh`). Railway builds and runs it.
2. Add a **Redis** service to the same project (Railway's Redis template). This is required — replicas coordinate through Redis, which is what makes horizontal scaling form one cluster instead of N disconnected servers.
3. On the LiveKit service → **Variables**, set:
 - `LIVEKIT_API_KEY` and `LIVEKIT_API_SECRET` — the pair from Step 0.
 - `REDIS_URL` — reference the Redis service's connection URL variable (Railway → the Redis service exposes a `REDIS_URL`; add it as a shared/referenced variable so LiveKit sees it over the private network).
4. On the LiveKit service → **Settings → Networking, Generate Domain** and set the target port to **7880**. That public domain, as `wss://<that-domain>`, is your `LIVEKIT_URL`.

Keep the LiveKit service single-region (the default). Then the scaler's `numReplicas` call scales it correctly. Only if you deliberately make it multi-region do you set `LIVEKIT_RAILWAY_REGION` (below).

Step 2 — Point the app at the LiveKit service

On your **app/backend** service → Variables, set (the token minter `sessionLiveKitToken` reads these):

- `LIVEKIT_URL` = `wss://<the-domain-from-step-1>`
- `LIVEKIT_API_KEY` = the **same** key from Step 0
- `LIVEKIT_API_SECRET` = the **same** secret from Step 0

These live in server env only — never in the client bundle.

Step 3 — Give the scaler its four values

The scaler talks to the Railway API. In your **admin settings** (or as env), set:

Setting	Value	Where to find it
LIVEKIT_SCALE_PROVIDER	railway	(was none)
RAILWAY_TOKEN	a Railway account or team token	Railway → Account/Team Settings → Tokens
LIVEKIT_RAILWAY_SERVICE_ID	the LiveKit service's id	Railway service URL, or the service Settings
LIVEKIT_RAILWAY_ENVIRONMENT_ID	the environment's id (e.g. production)	Railway env URL, or project Settings

LIVEKIT_SCALE_ENABLED is already **on** by default, so the moment these are set the every-minute controller starts setting the replica count for real.

Step 4 — Verify (before hosting is even on)

Call the controller in preview mode to see it decide without acting:

```
POST /functions/livekitScaleController { "dry_run": true }
```

You'll see `concurrent_viewers`, `desired_nodes`, and the Railway action it *would* take. With hosting still counsel-gated off there are no viewers, so it holds **0 nodes = \$0** — exactly right. When your attorney clears hosting and real viewers arrive, it scales the LiveKit service up node-by-node and back down to zero when they leave.

To smoke-test the scaling path itself before go-live, POST `{ "concurrent_viewers": 2500 }` (admin) and confirm it asks Railway for ~3 replicas, then `{ "concurrent_viewers": 0, "force_zero": true }` to confirm it scales back to zero.

Honest limits of Railway for live media

Railway is the simplest place to *scale replicas*, but it exposes **one public port per service**, which is a constraint for WebRTC:

- **Signaling** (the wss connection) rides that public HTTPS port — works cleanly.
- **Media** rides **ICE over TCP** on the RTC port (the config sets `tcp_port: 7881` and disables UDP, which Railway doesn't route). For screen-share and live-shopping this is fine. If you find media failing for viewers on restrictive networks, expose 7881 via a **Railway TCP Proxy** (Settings → Networking → TCP Proxy) or run a TURN-over-TLS sidecar.
- **Railway caps total replicas at 50.** `LIVEKIT_SCALE_MAX_NODES` (default 20) stays safely under it — don't raise it past ~45.
- For very heavy live-video at scale, a **cheap VPS pool** (Hetzner/OVH/Vultr) with real UDP is both more robust for media and cheaper on egress. The controller works identically there — you'd switch `LIVEKIT_SCALE_PROVIDER` to `webhook` and point it at a node-pool endpoint. Nothing else changes. Start on Railway; move the media tier to a VPS only if/when media quality or egress cost demands it.

Live hosting itself stays counsel-gated (`SESSION_HOSTING_ENABLED` off) — this deploy makes the media tier ready and self-scaling, it does not turn hosting on. See `COST-FLOOR-AND-LLAMA-ROUTING.md` and `LIVEKIT-HOSTING-COST-MODEL.md`.